UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE
CORSO DI LAUREA MAGISTRALE IN CONTROL SYSTEMS ENGINEERING

« NONLINEAR MODEL PREDICTIVE CONTROL »
INVERTED PENDULUM ON A CART

Course: Adaptive & Model Predictive Control
Instructor: Beghi Alessandro

Student:
Giacomo Vianello m. 2140028

Academic year 2025/2026

TABLE OF CONTENTS

1. **Introduction:** Goals & Solver Setup
2. **Task 1: Jerk Control** (Extended Formulation & Tuning)
3. **Task 2: Robustness** (Model Mismatch & Horizon Paradox)
4. **Task 3: LMPC vs. NMPC** (Performance & Stability Limits)
5. **Conclusions**

Access the GitHub Repo here.

INTRODUCTION

1. Introduction

2. Task 1

3. Task 2.3

4. Task 3

5. Conclusion

- **Goal:** Control an Inverted Pendulum on a Cart for swing-up and stabilization.
- **Analysis**
  - **Task 1:** Achieve smooth control inputs by penalizing force variation ($\dot{F}$)
  - **Task 2.3:** Analyze robustness against parameter mismatch ($\ell_{model} \neq \ell_{real}$)
  - **Task 3:** Compare Linear (LMPC) vs. Nonlinear (NMPC) formulations.
- **Tool:** Nonlinear Model Predictive Control (NMPC) using ACADOS (Python interface).

OCP			Controller
Integrator	**NLP Sovler**	**QP Solver**	**Sampling Time**
ERK	SQP	HPIMP (unless specified)	$T_S = 20$ ms
System not *stiff* $\sigma(A_{lin}) = \{0, 0, 3.7, -3.7\}$	Gauss-Newton Hessian approx	Partial Condensing	
	$NLP_{max\ iter} = 100$	$QP_{max\ iter} = 50$	

TASK 1: EXTENDED FORMULATION

1. Introduction

2. Task 1

3. Task 2.3

4. Task 3

5. Conclusion

TASK 1

Goals:

- Perform swing-up maneuver using the **augmented** system dynamics: $\boldsymbol{x}_{aug} = [p, \theta, v, \omega, F]^\top$.
- The control input is now the **jerk** (force derivative): $u = \dot{F}$.

Objective:

- Weight both the *magnitude* of the force (via q_F in Q) and its *variation* (via R)
- **How does the weighting terms impact on performance?**

$$\boldsymbol{x}_{aug}^\top Q \boldsymbol{x}_{aug} = q_p p^2 + q_\theta \theta^2 + q_v v^2 + q_\omega \omega^2 + q_F F^2, \qquad R u^2 = r\, \dot{F}^2$$

In the following:

1. *Aggressive Tuning:* $Q = \mathrm{diag}(10, 10, 0.1, 0.1, 10^{-3})$, $R = 10^{-3}$
2. *Smooth Tuning:* $Q = \mathrm{diag}(50, 100, 0.1, 0.1, 10^{-2})$, $R = 0.1$

TASK 1: AGGRESSIVE TUNING

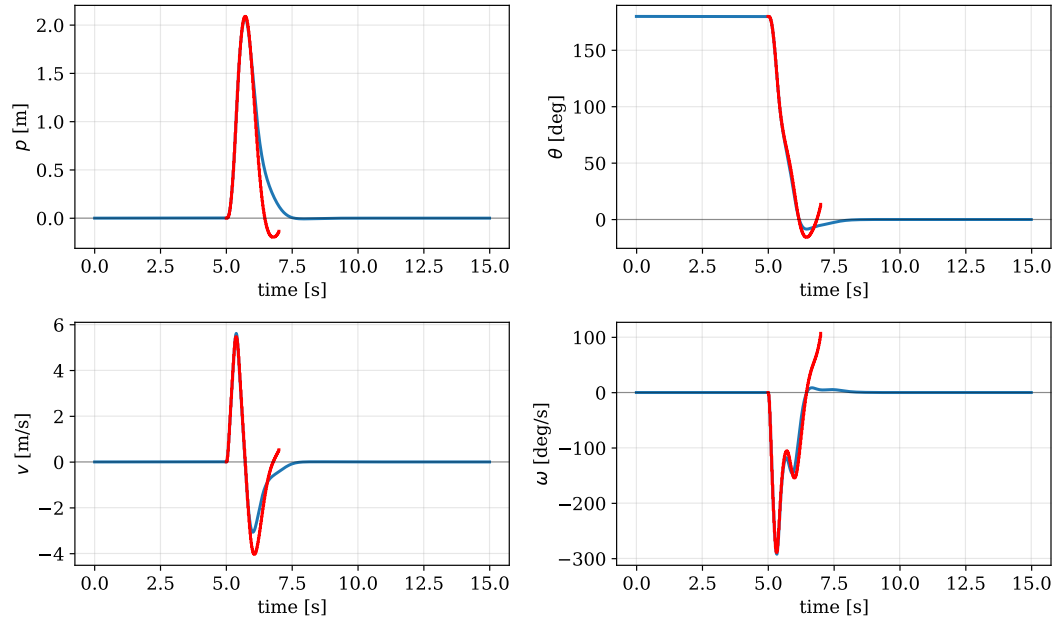
1. Introduction

2. Task 1

3. Task 2.3

4. Task 3

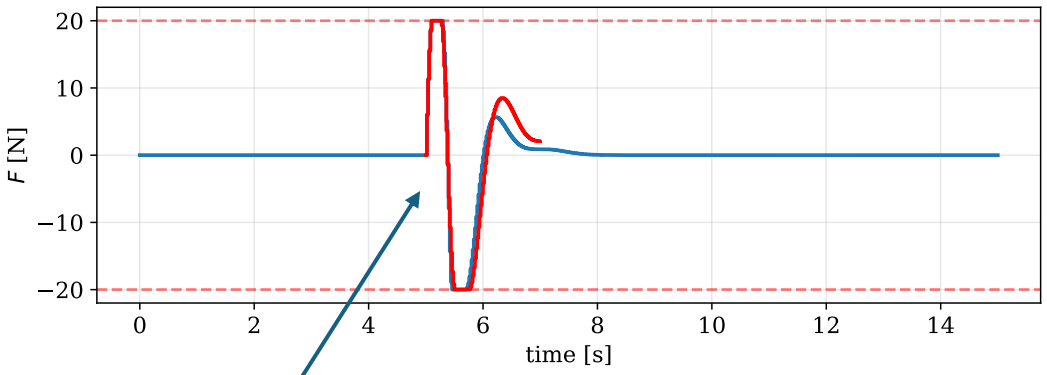
5. Conclusion



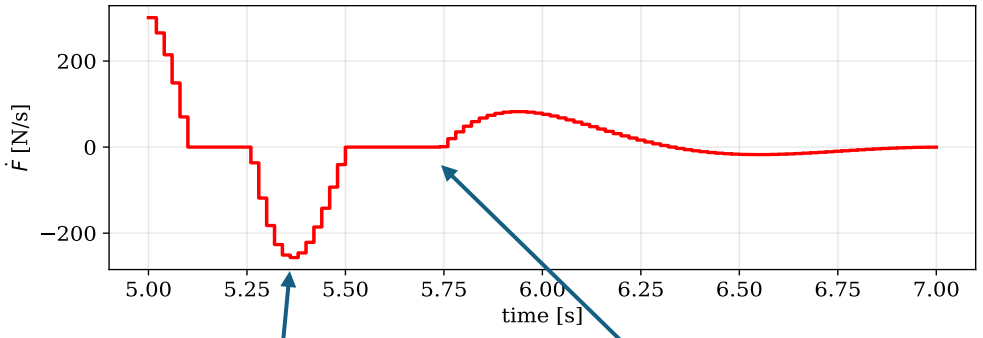
Weights:

$$Q = \mathrm{diag}(10, 10, 0.1, 0.1, 10^{-3}), \qquad R = 10^{-3}$$

Low penalty on input variation ($R = 10^{-3}$) allows the controller to drive the force rapidly to its limits. This **minimizes** the swing-up time but **increases** the mechanical stress ($\dot{F}_{max} \approx \pm 200$ N).



Bang-bang behavior



High mechanical stress

Saturation

TASK 1: SMOOTH TUNING

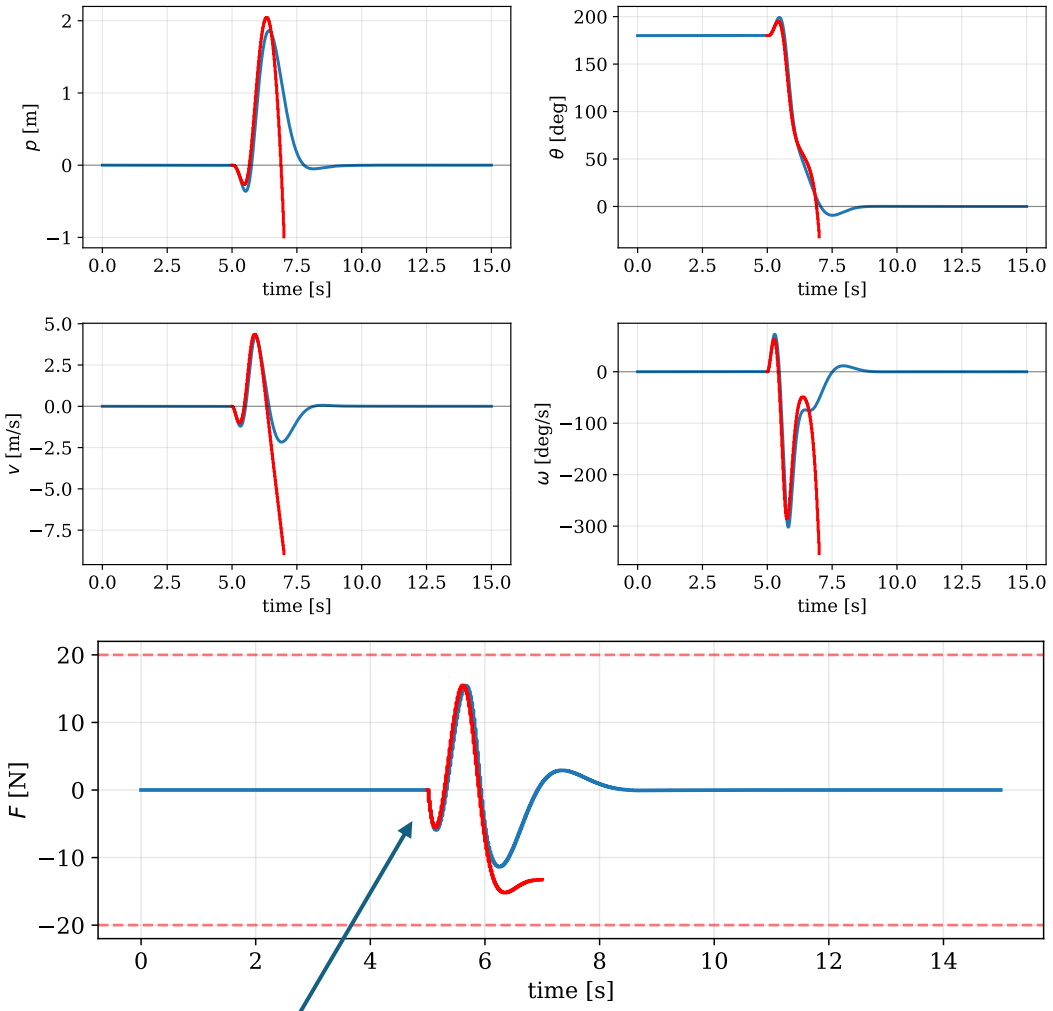
1. Introduction

2. Task 1

3. Task 2.3

4. Task 3

5. Conclusion



Smooth control action

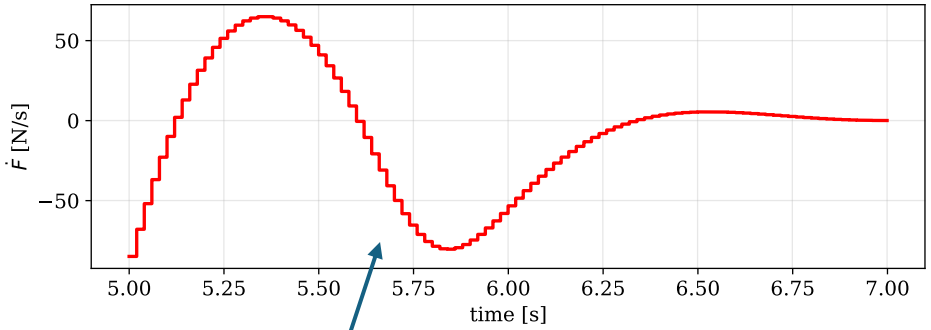
Weights:

$$Q = \mathrm{diag}(50, 100, 0.1, 0.1, 10^{-2}), \qquad R = 0.1$$

$\bar{t}_{CPU} = 0.56$ ms (aggressive)

$\bar{t}_{CPU} = 1.172$ ms (smooth)

High penalty on input variation ($R = 0.1$) forces the optimizer to plan a **smooth** trajectory. This **protects** the actuator but makes the **optimization problem harder** to solve.



Stress is 4x lower than before

TASK 1: SMOOTH TUNING CPU TIME

1. Introduction

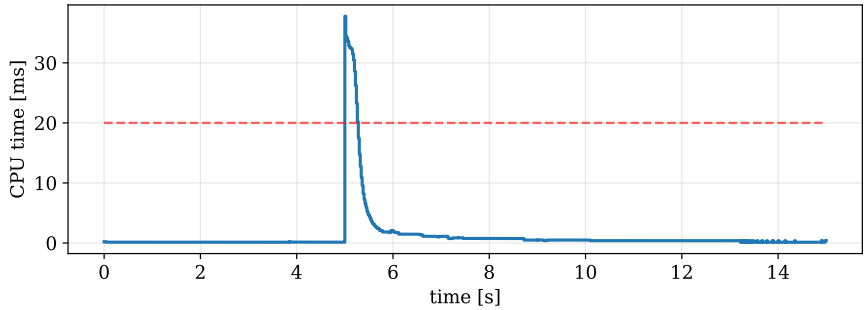
2. Task 1

3. Task 2.3

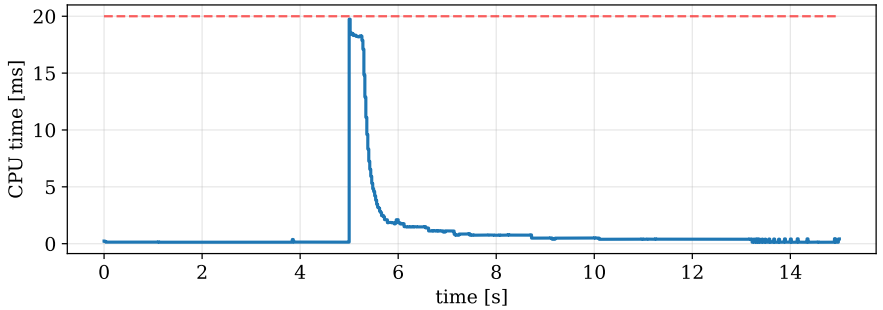
4. Task 3

5. Conclusion

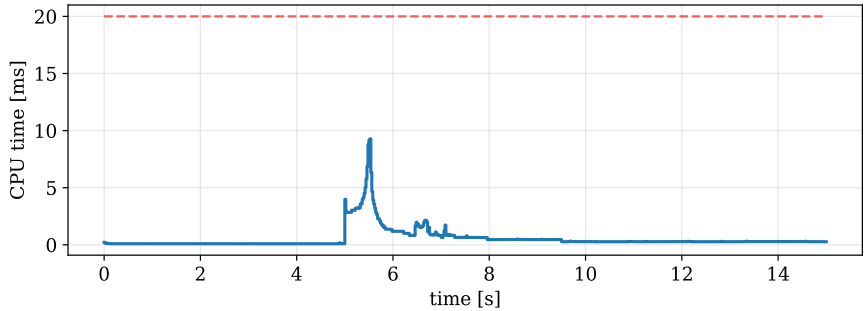
- In the «smooth tuning» scenario, the average time required to solve the NLP exceeds the controller sampling time ($T_S = 20$ ms) $\Longrightarrow$ **real-time** implementation becomes **impossible**!
- Some solutions:



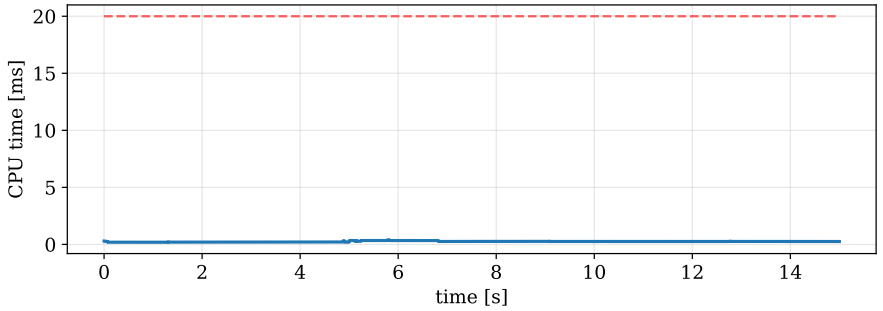
SQP: $N = 100$, $NLP_{\max iter} = 100$, $QP_{\max iter} = 50$ (HPIPM), $\bar{t}_{CPU} = 1.172$ ms



SQP: $N = 100$, $NLP_{\max iter} = 50$, $QP_{\max iter} = 50$ (HPIPM), $\bar{t}_{CPU} = 0.923$ ms



SQP: $N = 70$, $NLP_{\max iter} = 50$, $QP_{\max iter} = 5$ (warm-start, QPOASES), $\bar{t}_{CPU} = 0.51$ ms



SQP RTI: $N = 70$ (Partial HPIPM), $\bar{t}_{CPU} = 0.183$ ms

TASK 2.3: MODEL MISMATCH

1. Introduction

2. Task 1

3. Task 2.3

4. Task 3

5. Conclusion

TASK 2.3

Goals:

- Evaluate the NMPC robustness by introducing a model mismatch in the pendulum.
  - **True Plant:** $\ell_{true} = 0.9, 1.0$ m;
  - **Controller model:** $\ell = 0.8$ m
- The controller systematically **overestimates the system's natural frequency:** it "believes" the pendulum is shorter and faster than it actually is.

In the following:

- Effects of the tuning weights
  - Conservative tuning: $Q = \text{diag}(10, 10, 0.1, 0.1), \quad R = 0.1$
  - Aggressive tuning: $Q = \text{diag}(50, 100, 0.1, 0.1), \; R = 0.01$
- Paradox of prediction horizon N

TASK 2.3: CONSERVATIVE TUNING

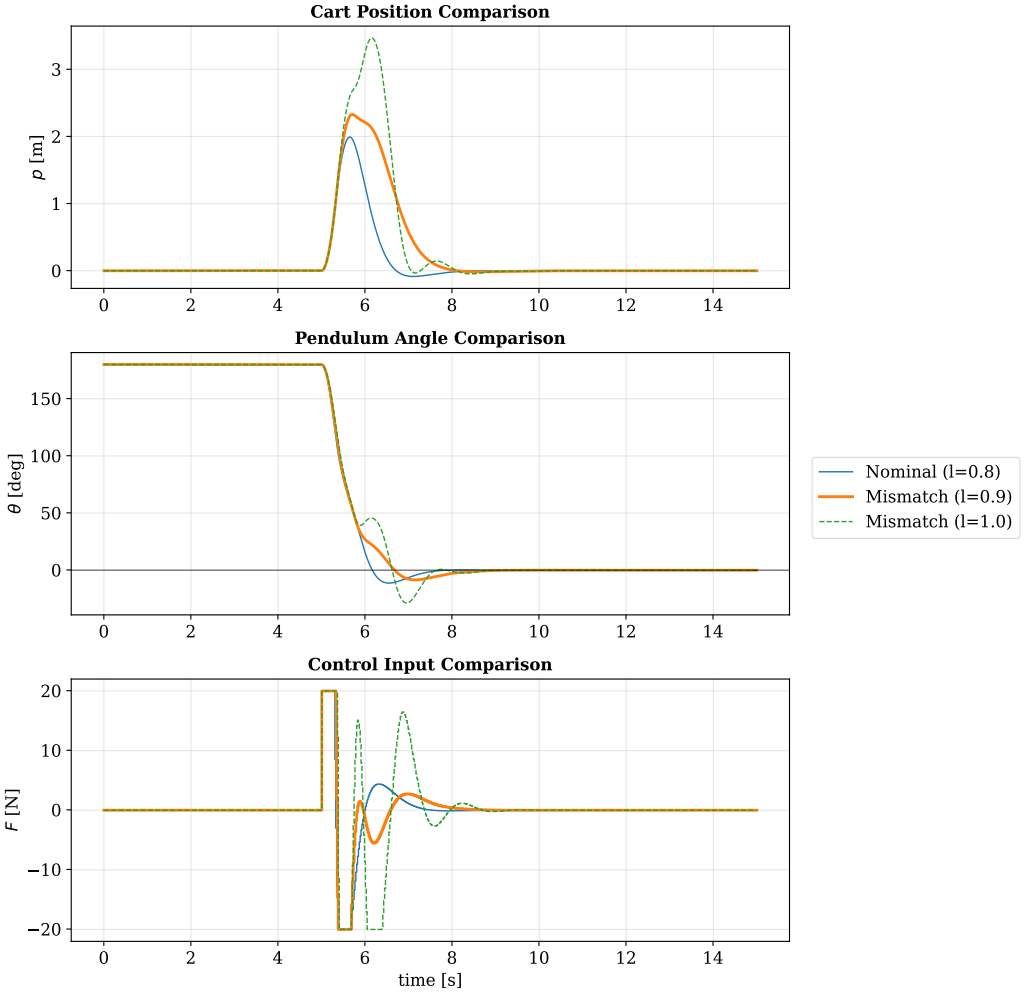
1. Introduction

2. Task 1

3. Task 2.3

4. Task 3

5. Conclusion



$Q = \text{diag}(10, 10, 0.1, 0.1)$, $R = 0.1$, $N = 100$:

- Real pendulum is slower and has more inertia than the model expects.
- Controller applies a force expected to swing up a shorter/lighter pendulum.
- When real pendulum lags behind, controller is slow to react due to the conservative tuning, leading to the large transient error.
- Cart position exhibits a massive overshoot in the mismatch case.

Conservative tuning guarantees **stability** even with significant parameter mismatch (+20%), but at the expense of **performance** (larger space required).
The system is **robust**, but **sluggish**.

TASK 2.3: AGGRESSIVE TUNING

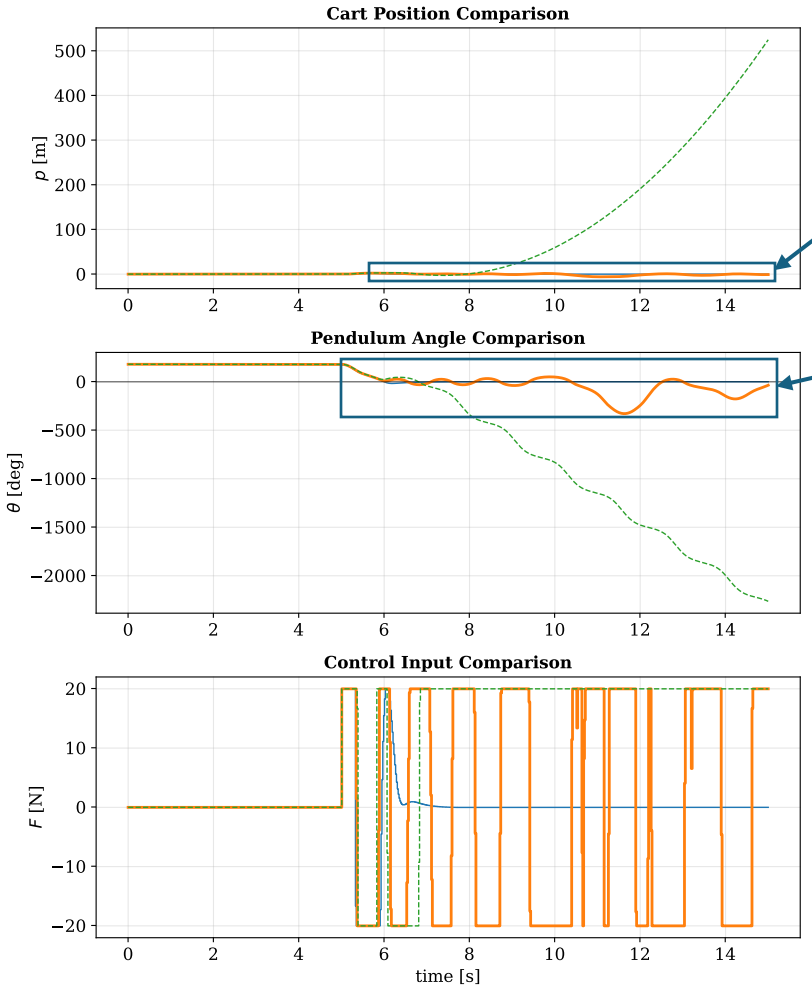
1. Introduction

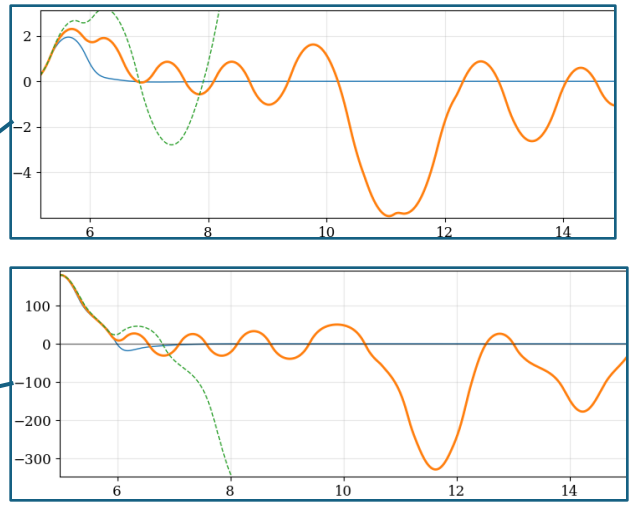
2. Task 1

3. Task 2.3

4. Task 3

5. Conclusion





$Q = \text{diag}(50, 100, 0.1, 0.1)$, $R = 0.01$, $N = 100$:

- A **long horizon**, $N = 100$, relies heavily on the internal model to plan complex, long-term maneuvers.
- Solver finds a trajectory that is optimal for the *nominal* model ($\ell = 0.8$) but inconsistent with the *real* plant ($\ell_{true} = 0.9, 1.0$).
- Swing-up relies on resonance ($\sqrt{g/l}$). Since $\ell_{true} \neq \ell$, the planned energy pumping is **out of phase** with the real pendulum.
- Controller **fights** the system's dynamics, leading to instability.

TASK 2.3: PARADOX OF PREDICTION HORIZON

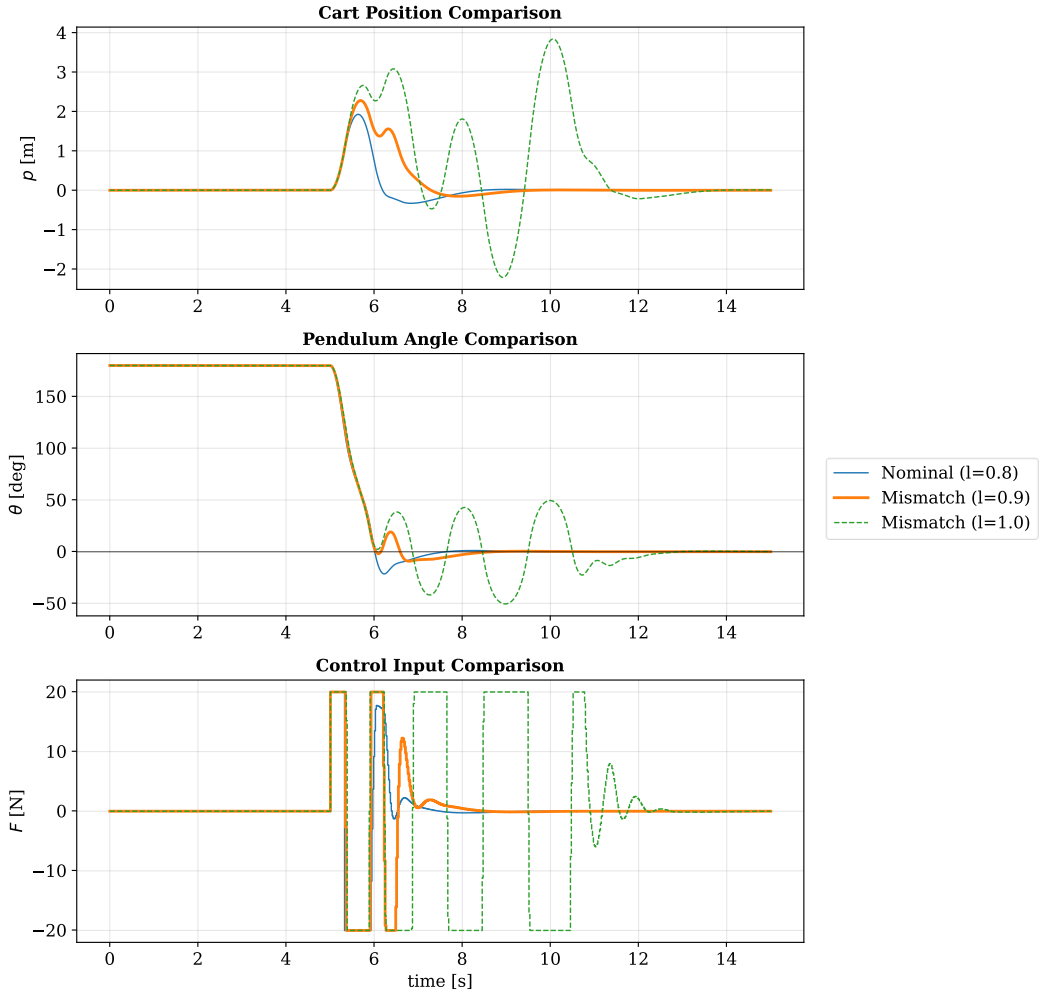
1. Introduction

2. Task 1

3. Task 2.3

4. Task 3

5. Conclusion



$Q = \text{diag}(50, 100, 0.1, 0.1)$, $R = 0.01$:

- With a **shorter prediction horizon**, $N = 40$, the controller is able to stabilize the pendulum in the upward position, even in presence of model mismatch.
- Why?
  - A short horizon makes the controller **myopic**. It cannot plan far ahead, so it focuses entirely on reducing the **immediate** error.
  - We force the controller to be more **reactive** than **predictive**.

TASK 2.3: PARADOX OF PREDICTION HORIZON

1. Introduction

2. Task 1

3. Task 2.3

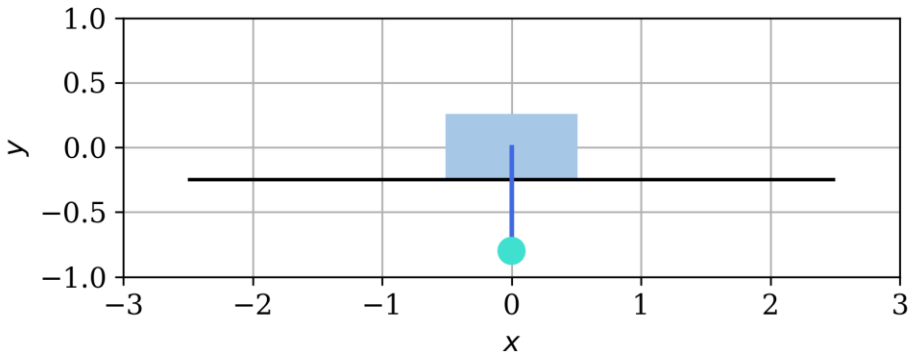
4. Task 3

5. Conclusion

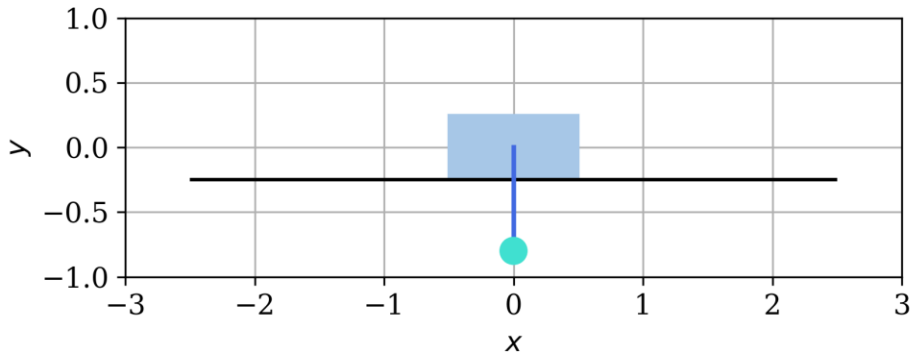
Conclusion: in the presence of model mismatch, especially with aggressive tuning,

«Long horizons optimize the wrong future,

short horizons optimize the true present»



$N = 100, \ell_{true} = 0.9\,\text{m}$



$N = 40, \ell_{true} = 0.9\,\text{m}$

TASK 3: LMPC vs. NMPC

1. Introduction

2. Task 1

3. Task 2.3

4. Task 3

5. Conclusion

TASK 3

Goals:

- Evaluate the differences in performance between NMPC and LMPC in a position-tracking scenario.
- Pendulum starts in the upright equilibrium. We command the cart to move back and forth (square wave reference) while keeping the pole balanced.
- Selected a specific set of weights (Q) that penalizes position error heavily ($q_p = 500$) to force the system to move, but keeps a moderate penalty on the angle ($q_\theta = 300$) to prevent falling.

$$Q = \text{diag}(500, 300, 1, 1), \quad R = 0.1$$

- The horizon is initially set to $N = 40$, which corresponds to a 0.8-second prediction window.

TASK 3: LMPC vs. NMPC

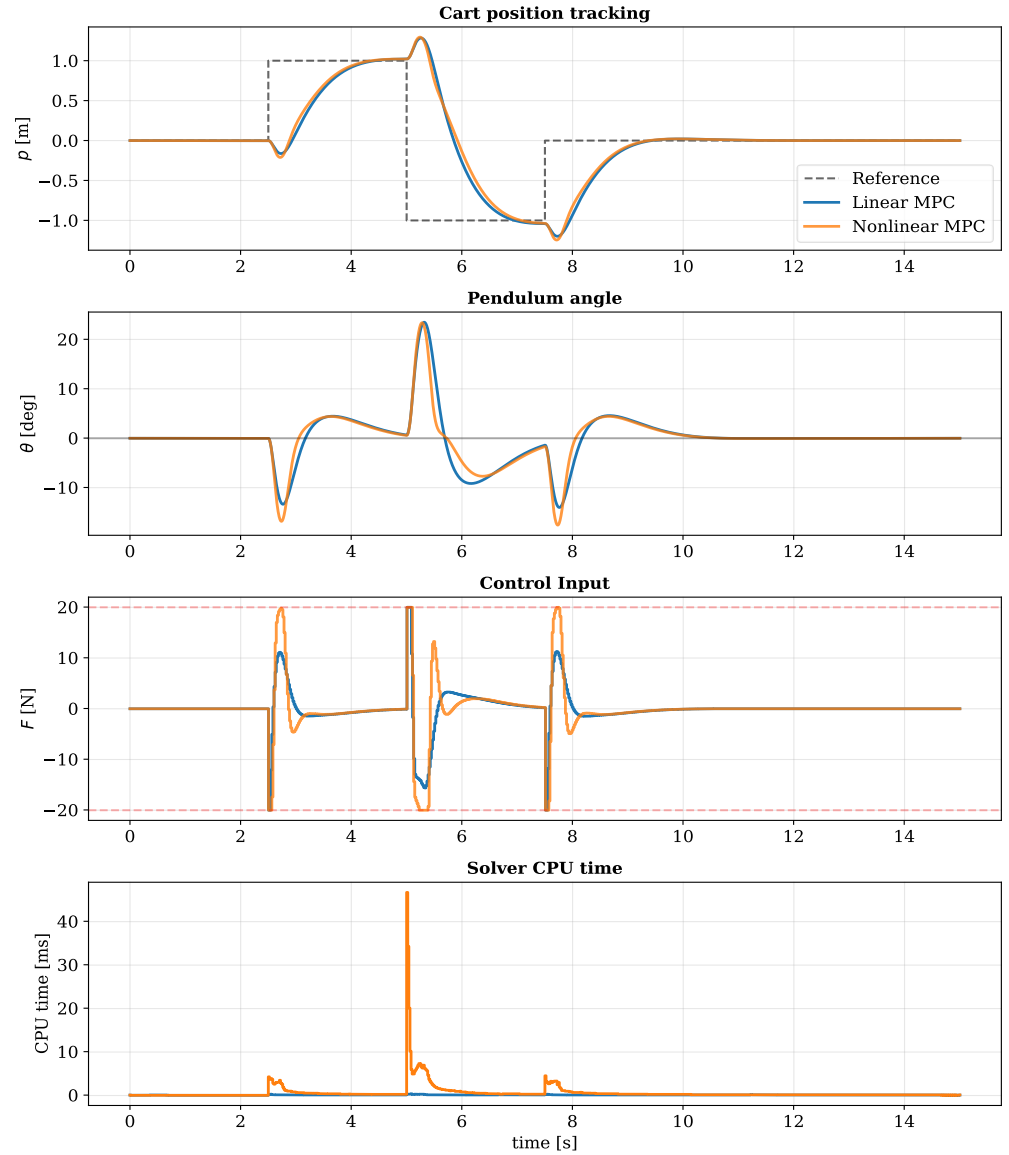
1. Introduction

2. Task 1

3. Task 2.3

4. Task 3

5. Conclusion



- With appropriate tuning ($Q = \mathrm{diag}(500, 300, 1, 1)$, $R = 0.1$) and a reasonably **short** prediction horizon ($N = 40$), LMPC and NMPC perform quite the **same**.
- Control action keeps $\theta \in [-20°, 20°]$: linear approximation $\sin(\theta) \approx \theta$ is valid.
- Both NMPC and LMPC are able to track the position reference while maintaining the pendulum in the upward equilibrium.
- However, LMPC **outperforms** NMPC in terms of:
  - **Control input energy**: LMPC saturates the force input only when the cart has to be moved from $x = 1$ m to $x = -1$ m.
  - **Computational time**: Since LMPC solves the NLP with a maximum number of SQP iterations set to 1 (QP), it is way faster ($\approx$ 7x) than its nonlinear counterpart:

$$\bar{t}_{LMPC} \approx 0.1 \text{ ms}, \bar{t}_{NMPC} \approx 0.7 \text{ ms}.$$

- LMPC **cannot** be used for swing-up maneuver.

TASK 3: LMPC vs. NMPC and Prediction Horizon

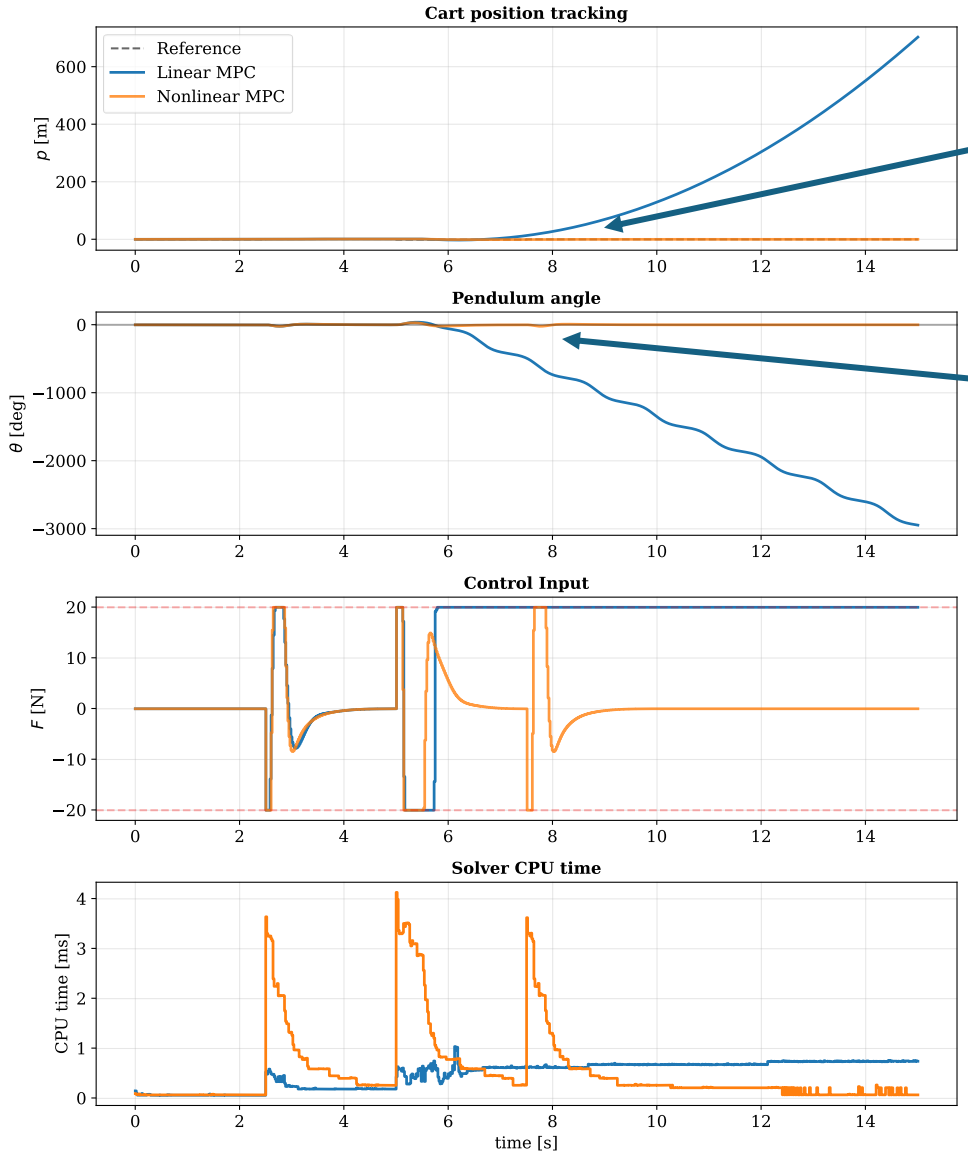
1. Introduction

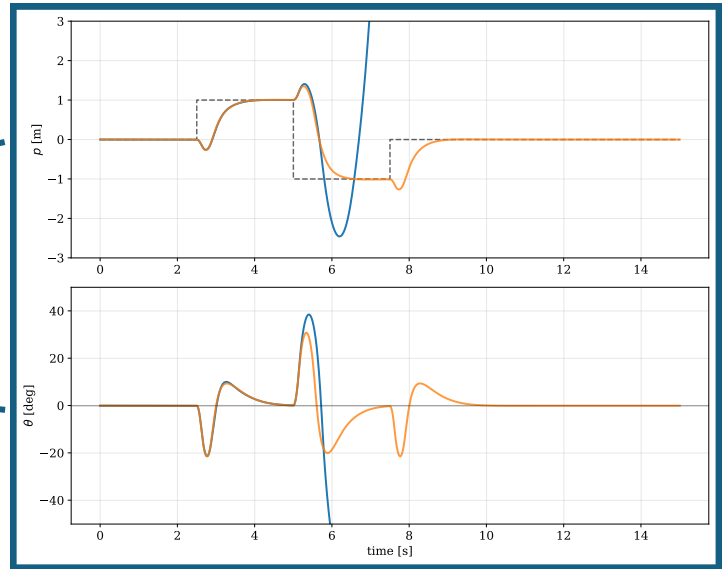
2. Task 1

3. Task 2.3

4. Task 3

5. Conclusion





- LMPC uses a linearized model that assumes $\sin\theta = \theta$.
- The further you look into the future, the more the 'predicted' linear dynamics drift away from the 'real' nonlinear physics.
- $N = 60$: LMPC optimizer 'trusts' its linear model too much. Inputs are slightly wrong and over a long window these small errors accumulate, leading the controller to push the system into instability.
- $N = 40$: prevents this drift by forcing the controller to rely on «immediate» feedback.

CONCLUSIONS

1. Introduction

2. Task 1

3. Task 2.3

4. Task 3

5. Conclusion

- **Smoothness vs. Speed (Task 1):** Augmenting the state to penalize Jerk ($u = \dot{F}$) successfully reduces mechanical stress, though it requires careful tuning of Q and R. Solver options have to be tweaked in order to lower the computational time of OCP ($\bar{t}_{CPU} < T_S$).
- **The Horizon Paradox (Task 2.3):** Counter-intuitively, **shorter horizons** ($N = 40$) **can be more robust** to model mismatch. Long horizons ($N = 100$) optimize a "false future" based on the wrong model, leading to instability.
- **LMPC Limits (Task 3):** LMPC is highly efficient ($\approx$ 7x faster than NMPC) and effective near equilibrium. However, it **diverges for large horizons** or aggressive maneuvers where the linear approximation ($\sin(\theta) \approx \theta$) fails.

Thank you for your attention.